WATERLOO ROCKETRY

Next-Gen Ground Support Software System (Orion)

Preliminary Design Review

Qian Qian
Software Lead

Shrey Shingala
Software Lead

October 29, 2026

Agenda

01 Overview, Motivation & Feasibility

02 Requirements

03 Explanation of Key Technological Choices

04 Data Modeling

05 Deployment Architecture

06 Testing

07 Project Management

08 Action Items

SECTION 01

Overview, Motivation & Feasibility

01

WATERLOO ROCKETRY

Overview

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Motivation

- Source-level breaking changes currently propagate across the stack, often requiring coordinated changes to dashboards and other consumers.
- Data contracts and system state are spread across multiple components, making behaviour harder to reason about and changes harder to validate.
- Adding new data sources and tooling increases maintenance overhead under the current architecture.
- We want a simpler, more reliable stack with stronger guarantees around shared data structures and interfaces.

Feasibility

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

SECTION 02

Requirements

02

WATERLOO ROCKETRY

Requirements Overview

The GitHub issues `2026-2027-software-issues#51` contains and is the root parent of issues containing:

- general technical directions
- design (tasks and decisions)
  - specification
  - API Protocol and Data Model
- implementation (requirements and tasks)

Please review its content.

General Requirements

Requirement	Motivation
Omnibus MUST remain production-ready until Orion is fully production-ready and is fully replacing Omnibus.	Allow smooth transition and ensure fallback option exists while issues are ironed out.
Orion SHOULD provide one authoritative API server for telemetry, shared state and command routing.	Provides a stable boundary between sources and consumers, and avoid system-wide breaks when a source or consumer is updated.
Public interfaces SHOULD be typed, versioned, documented and machine-checked.	Standardized interfaces to ensure safe interaction and validation of data.
Complete and well organized documentation MUST be provided for all public interfaces, including API, data model, and configuration.	Ensure that the system is usable and mainytainable by future developers and users.
The deployment of the system MUST be reproducible.	Ensure that the system can be deployed in a consistent manner.
The system MUST be easy to configure and the configuration methods MUST be documented.	Ensure that the system can be configured by all users. (Abandoning the current Python-script-based configuration system.)

SECTION 03

Explanation of Key Technological Choices

03

WATERLOO ROCKETRY

Go Stack

Replacing the current Python stack with a Go stack.

- It is a **typed** language, aligning with our goal of having typed messages schemas and interfaces.
- It compiles to a binary, which
  - is less dependent on the runtime environment,
  - is easier to deploy, and
  - is easier to distribute.

Go Stack - Key Changes

- Central Processing Server: switching from Python to Go
- Sink and Source modules: switching from Python to Go
- Dependency Management: switching from `uv` to Go's own dependency management tool.
- Testing Framework: switching from `pytest` to Go's built-in testing framework.
  - We might consider using other more feature-rich testing frameworks if it's deemed necessary during development.
- Networking: switching from WebSockets to Protobuf messages over gRPC,
  - it allows for typed messages, and
  - it supports both unary (request-response) and streaming operations, which is useful for supporting both command/info routing and telemetry data streaming.

Deployment

We continue to deploy on Raspberry Pi.

- Raspberry Pi OS, Docker, and Ansible are kept as they allow the deployment to be reproducible and can be used to redeploy the system quickly and consistently.
- Docker container may be further simplified compared to the Python stack, as the new stack is less dependent on the runtime environment and has fewer dependencies.
- Deliver a single binary, and use different CLI flags to configure the binary to run as different modules (e.g. server, sink, source, etc.), which is easier to deploy and maintain.

Launch App

Launch App will be a desktop app for use during rocket launches, replacing the old Omnibus Dashboard / DAQms. It will use **Electron** with **React and TypeScript**.

- Functional modern web development stack with easy compatibility on any operator's device.
- Easy to build, easy to deploy, easily compatible with the rest of our software.
- Compatible with Parsley through Parsley-TS (TypeScript).
- React is a well-known and well-supported frontend framework and is widely-used.

Data Studio

We are building Data Studio as a intended replacement for the current Omnibus “Data Processing” scripts, which are not well maintained and not intuitive to use.

Goal: a graphical app to review logs and export data, without needing to figure out how to use command-line scripts.

- **Import** Omnibus, Orion and other agreed log formats.
  - For Orion, it strictly follows the Orion data model and API.
- **Plot & Select** series and time ranges interactively.
- **Export** the selection as CSV, images, or other formats.

Data Studio

We want to use a modern web stack (e.g. React + TypeScript + D3.js) for Data Studio frontend, and use Go for the logic and data processing backend, which stays consistent with the rest of Orion system. `Wails` is the proposed framework for this stack.

- We might consider Vue for the frontend.
- Wails is sort of Electron, but it uses Go as the backend.
  - It is considered usable for the forseeable future and is actively maintained.
  - The newest version (v3.0.0) just released October 26th, 2026.
    The previous version (v2.17.0) was released mid-October 2026.
  - It was actively maintained with regular commits.
  - Unfortunately, it is not as popular and no public record shows that well-known companies or projects are using or backing it.

A dedicated PDR for Data Studio may be planned for the near future.

SECTION 04

Data Modeling

04

WATERLOO ROCKETRY

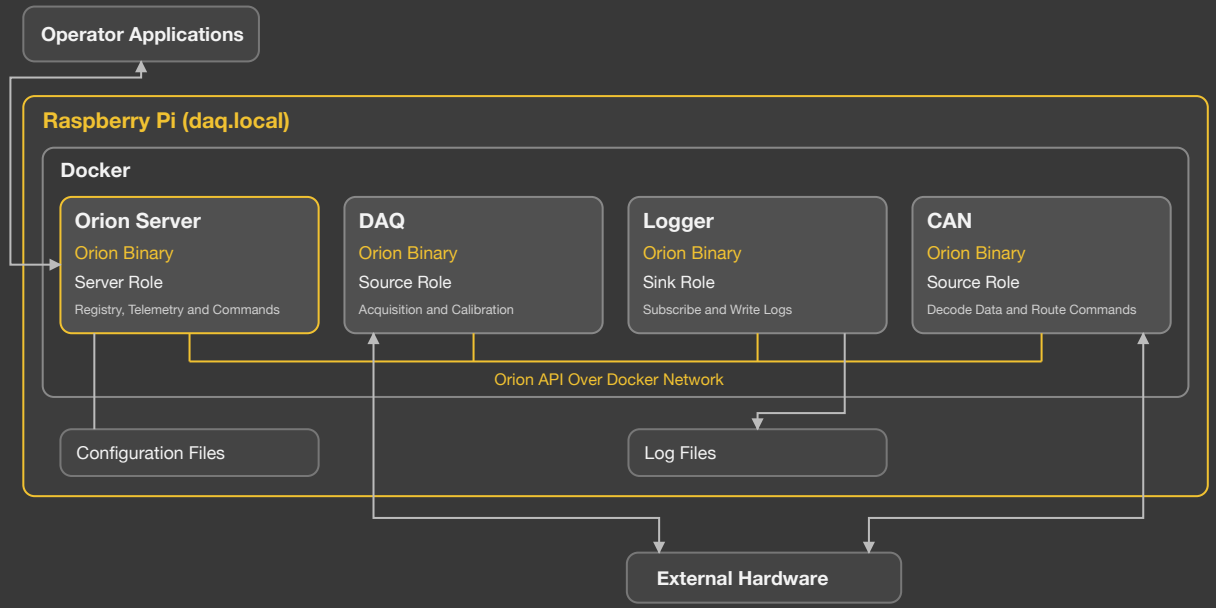
SECTION 05

Deployment Architecture

05

WATERLOO ROCKETRY

Raspberry Pi Deployment



SECTION 06

Testing

06

WATERLOO ROCKETRY

Testing

TL;DR: Everything gets unit tests.

- Keep intended behaviour well-defined for future maintainers, so we can fix bugs and add functionality without breaking existing workflows.
- Aim for **85% test coverage**, with explicit tests for invalid input, disconnects and failure handling.
- Add integration tests across the stack, including the server, sources, sinks, and applications.
  - Use Fake Source and replay for repeatable tests without physical hardware.
- Automate end-to-end testing where practical; the tooling and scope are still to be determined.
  - Do manual testing and validation before accepting operational changes.
- Well-maintained dependencies shall be used, and their versions pinned, so builds and test results are reproducible.

SECTION 07

Project Management

07

WATERLOO ROCKETRY

Timelines, and Reviews

`2026-2027-software-issues#51` contains the timeline and milestones for the Orion project.

Duration: About 9 Months, from Early October 2026 to Mid July 2027.

- Design and Specifications: October - Mid November;
- Prototype Work through Late November.
- Adapters, Deployment and Migration Preparation: Start in November
  - Continue Alongside Implementation.
- Native Implementation: Early January - Late June; CDR: Late January - Early February.
- Full Integration and Readiness Checks: Late February - Mid July.

SECTION 08

Action Items

08

WATERLOO ROCKETRY

Action Items

1. Review the issues in `2026-2027-software-issues#51` and its children.
2. Start working on the specification, including the definition of the communication protocol, data model, API, and configuration format.
3. Start working on the prototype, including the server, source and sink modules.
4. Start building LJM library in Go.
5. Plan for implementation of the Omnibus adapter which should allow Omnibus sources and sinks to work with Orion server.